Homework 3: Classification

Ariv Ahuja

DS4400 - Machine Learning

Code Link: `https://github.com/ArivAhuja1/DS4400/blob/main/HW3/HW3_code.ipynb`

[C] Problem 1: Logistic Regression (25 points)

We use `sklearn`'s `LogisticRegression` to train and test on the SPAMBASE dataset (4601 samples, 57 features). The data is split 75%/25% for training (3450 samples) and testing (1151 samples), and features are standardized using `StandardScaler`.

1.1: Confusion Matrix, Accuracy, Error, Precision, Recall, F1

Confusion Matrix (on the testing set):

$$\begin{bmatrix} 650 & 26 \\ 63 & 412 \end{bmatrix} \quad \text{TN} = 650,\ \text{FP} = 26,\ \text{FN} = 63,\ \text{TP} = 412$$

Metric	Value
Accuracy	0.9227
Error	0.0773
Precision	0.9406
Recall	0.8674
F1 Score	0.9025

Table 1: Logistic regression metrics on the testing set.

The model achieves 92.27% accuracy. Precision (0.9406) is higher than recall (0.8674), meaning the model is conservative in predicting spam—when it does predict spam, it is usually correct, but it misses some spam emails.

1.2: Feature Coefficients

The top 10 features by absolute coefficient magnitude are:
Positively correlated with SPAM (higher values ⇒ more likely spam): `char_freq_$` (1.1551), `word_freq_000` (0.9283), `word_freq_free` (0.9209), `word_freq_3d` (0.8517), `word_freq_remove` (0.8460). These are typical spam-associated words and symbols (dollar signs, "free", "remove").
Negatively correlated with SPAM (higher values ⇒ more likely ham): `word_freq_george` (−4.2499), `word_freq_hp` (−2.6773), `word_freq_project` (−1.4379), `word_freq_cs` (−1.4202), `word_freq_meeting` (−1.2160). These are work-related terms specific to the data collector's legitimate emails (George, HP, project, cs, meeting).

Sign	Feature	Coefficient
−	word_freq_george	−4.2499
−	word_freq_hp	−2.6773
−	word_freq_project	−1.4379
−	word_freq_cs	−1.4202
−	word_freq_meeting	−1.2160
+	char_freq_$	+1.1551
−	word_freq_edu	−1.1446
−	word_freq_conference	−1.0389
+	word_freq_000	+0.9283
−	word_freq_lab	−0.9272

Table 2: Top 10 features by absolute coefficient.

1.3: Varying Decision Threshold

Threshold T	Accuracy	Precision	Recall
0.25	0.9123	0.8502	0.9558
0.50	0.9227	0.9406	0.8674
0.75	0.8836	0.9547	0.7537
0.90	0.8280	0.9727	0.6000

Table 3: Metrics at different decision thresholds.

As the threshold T increases from 0.25 to 0.9:

- **Precision increases** (0.8502 → 0.9727): requiring higher confidence means fewer false positives.
- **Recall decreases** (0.9558 → 0.6000): a stricter threshold misses more actual spam.
- **Accuracy** peaks at $T = 0.50$ (0.9227) and decreases for extreme thresholds, reflecting the trade-off between precision and recall.

This demonstrates the classic precision-recall trade-off controlled by the decision threshold.

[C] Problem 2: Gradient Descent for Logistic Regression (25 points)

We implement gradient descent for logistic regression from scratch. The update rule is:

$$\theta \leftarrow \theta - \alpha \cdot \frac{1}{m} X^T(\sigma(X\theta) - y)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function. We test learning rates $\alpha \in \{0.01, 0.1, 1.0\}$ and record the cross-entropy loss at iterations 10, 50, and 100.

Cross-Entropy Loss at Various Iterations

Higher learning rates converge faster: $\alpha = 1.0$ achieves a loss of 0.2321 at 100 iterations, while $\alpha = 0.01$ has only reached 0.4687. All three are still decreasing, indicating convergence has not been fully reached for the smaller learning rates.

Learning Rate	Iter 10	Iter 50	Iter 100
0.01	0.6512	0.5416	0.4687
0.10	0.4654	0.3241	0.2891
1.00	0.2853	0.2435	0.2321

Table 4: Cross-entropy loss during gradient descent.

Metrics at 100 Iterations

LR (α)	Accuracy	Precision	Recall	F1
		Training Set		
0.01	0.8977	0.8734	0.8610	0.8671
0.10	0.9078	0.9113	0.8445	0.8766
1.00	0.9209	0.9236	0.8677	0.8948
		Testing Set		
0.01	0.9036	0.9009	0.8611	0.8805
0.10	0.9062	0.9238	0.8421	0.8811
1.00	0.9157	0.9315	0.8589	0.8938

Table 5: GD logistic regression metrics at 100 iterations.

Comparison with sklearn:

- sklearn Train: Acc=0.9258, Prec=0.9240, Rec=0.8812, F1=0.9021
- sklearn Test: Acc=0.9227, Prec=0.9406, Rec=0.8674, F1=0.9025

The sklearn model (which runs until convergence with L2 regularization) slightly outperforms all GD variants. The $\alpha = 1.0$ GD model comes closest (test accuracy 0.9157 vs. 0.9227 for sklearn). With more iterations, the GD implementation would converge closer to the sklearn results. The gap is due to (1) limited iterations (100), and (2) sklearn's L2 regularization which helps generalization.

[C] Problem 3: Comparing Classifiers (25 points)

We train Logistic Regression, LDA, and kNN on the same train/test split from Problem 1.

3.1: kNN Cross-Validation for k

We use 5-fold cross-validation on the training set to select the best k:

Best $k = 7$ with CV error $= 0.0933$. As k increases beyond 7, recall drops significantly while precision stays relatively stable, indicating the model becomes too conservative. Very small k values lead to overfitting.

3.2: Classifier Comparison

Observations:

k	**Accuracy**	**Error**	**Precision**	**Recall**
1	0.9014	0.0986	0.8777	0.8670
3	0.9009	0.0991	0.8825	0.8595
5	0.9026	0.0974	0.8906	0.8543
7	0.9067	0.0933	0.8963	0.8595
9	0.9049	0.0951	0.8994	0.8505
11	0.9006	0.0994	0.8977	0.8401
15	0.8988	0.1012	0.9017	0.8303
21	0.8907	0.1093	0.9002	0.8087
31	0.8829	0.1171	0.8933	0.7945
51	0.8791	0.1209	0.9036	0.7713

Table 6: kNN cross-validation results for various k.

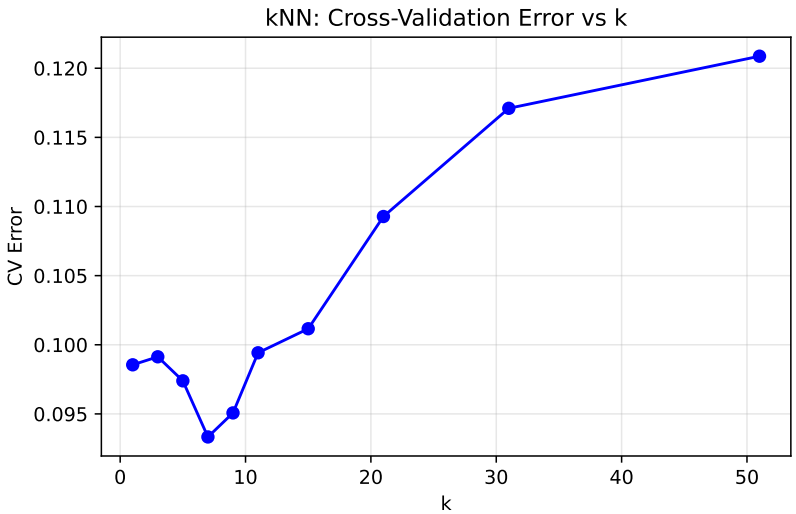

Figure 1: kNN cross-validation error vs. k.

- **Best performer:** Logistic Regression achieves the highest test accuracy (0.9227) and the best balance of precision (0.9406) and recall (0.8674).
- **Worst performer:** LDA has the lowest accuracy on both training (0.8867) and testing (0.8853) sets, with notably low recall (0.7811), suggesting it misses many spam emails. LDA assumes features are normally distributed with shared covariance, which may not hold well for these word-frequency features.
- kNN (k=7) performs well on training data (0.9226) but drops to 0.9018 on test data, showing some overfitting. It is non-parametric and does not make distributional assumptions, but is sensitive to the curse of dimensionality with 57 features.

3.3: ROC Curve (sklearn)

The AUC of **0.9747** indicates excellent discriminative ability. The curve hugs the top-left corner, showing the model can achieve high true positive rates with low false positive rates across most thresholds.

Model	Accuracy	Error	Precision	Recall
Training Set				
Logistic Regression	0.9258	0.0742	0.9240	0.8812
LDA	0.8867	0.1133	0.9157	0.7795
kNN (k=7)	0.9226	0.0774	0.9167	0.8804
Testing Set				
Logistic Regression	0.9227	0.0773	0.9406	0.8674
LDA	0.8853	0.1147	0.9298	0.7811
kNN (k=7)	0.9018	0.0982	0.9114	0.8442

Table 7: Comparison of all 3 classifiers on training and testing sets.

3.4: Manual ROC Curve

We compute FPR and TPR manually for thresholds $T \in \{0, 0.1, 0.2, \ldots, 1.0\}$:

Threshold	FPR	TPR
0.0	1.0000	1.0000
0.1	0.3077	0.9874
0.2	0.1524	0.9684
0.3	0.0917	0.9389
0.4	0.0636	0.9095
0.5	0.0385	0.8674
0.6	0.0340	0.8295
0.7	0.0266	0.7874
0.8	0.0222	0.7179
0.9	0.0118	0.6000
1.0	0.0000	0.0021

Table 8: Manually computed FPR and TPR at each threshold.

Differences: The sklearn ROC curve is smooth because it evaluates every unique predicted probability as a threshold (hundreds of points), while the manual curve uses only 11 evenly-spaced thresholds, resulting in a coarse, piecewise-linear approximation. The manual ROC slightly underestimates the true AUC because it misses the fine-grained threshold values where the curve bends. **To make the two curves more similar**, we could use more threshold values (e.g., 100 or 1000 evenly spaced values, or use every unique predicted probability as a threshold), which would produce a curve that closely matches the sklearn output.

[C] Problem 4: Cross Validation (25 points)

4.1: Custom k-Fold Cross-Validation Implementation

We implement k-fold CV from scratch:

1. Randomly shuffle and divide the data into k equal partitions.

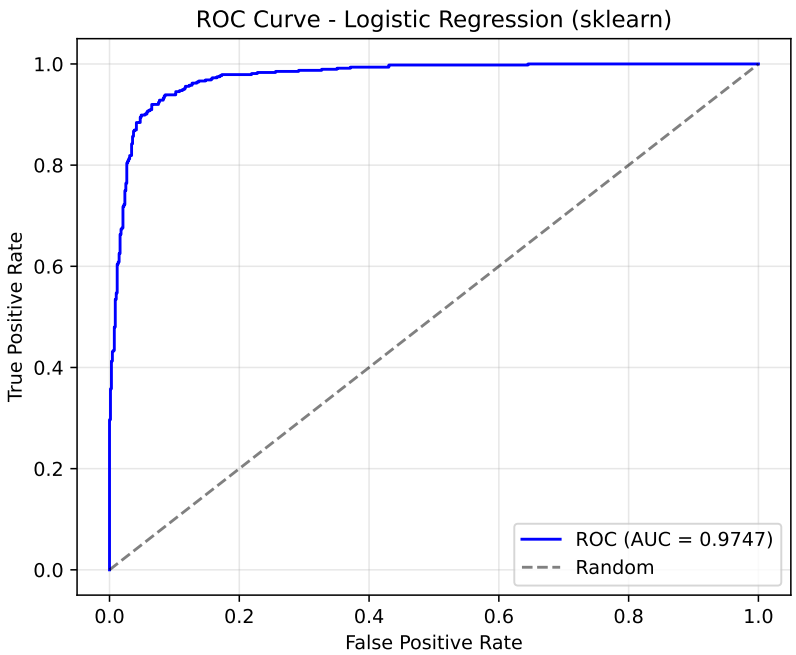

Figure 2: ROC curve for logistic regression (sklearn), AUC = 0.9747.

2. For each fold $i \in \{1, \ldots, k\}$: train on $k-1$ partitions, test on partition i.
3. Standardize features within each fold (fit on training partitions, transform validation).
4. Record the validation error for each fold and compute the average.

4.2: CV Results for Logistic Regression and LDA

k	**Model**	**Fold Errors**	**Avg Error**
2*5	Logistic Regression	[0.0815, 0.0761, 0.0793, 0.0717, 0.0706]	0.0759
	LDA	[0.1185, 0.1065, 0.1196, 0.1065, 0.1118]	0.1126
2*10	Logistic Regression	[0.0739, 0.0717, 0.0717, 0.0804, 0.0913, 0.0609, 0.0848, 0.0543, 0.0717, 0.0759]	2*0.0737
	LDA	[0.1174, 0.1130, 0.1065, 0.1065, 0.1217, 0.1087, 0.1283, 0.0913, 0.1217, 0.1150]	2*0.1130

Table 9: Custom k-fold CV results.

4.3: Comparison

Logistic Regression performs better than LDA across both $k = 5$ and $k = 10$:

- At $k = 5$: LR avg error = 0.0759 vs. LDA avg error = 0.1126.
- At $k = 10$: LR avg error = 0.0737 vs. LDA avg error = 0.1130.

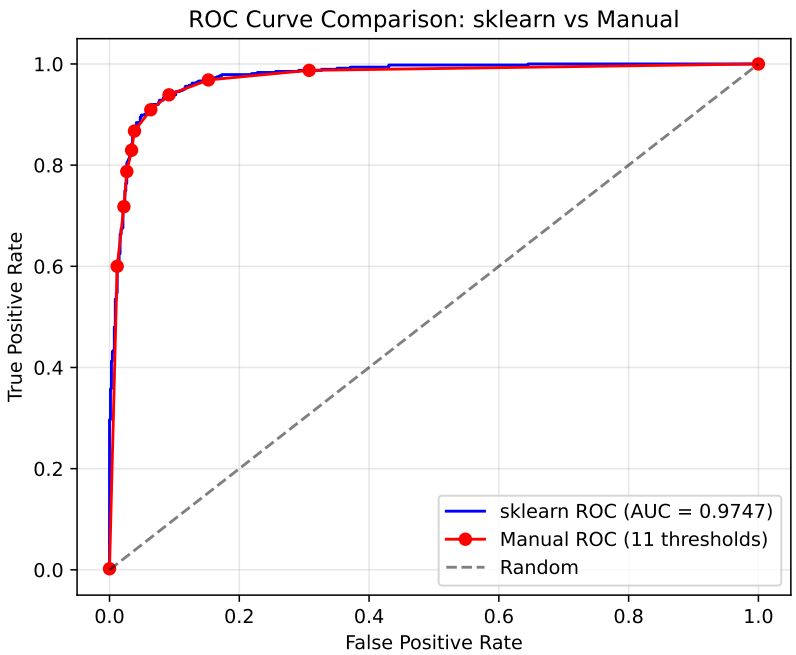

Figure 3: ROC curve comparison: sklearn (smooth) vs. manual (11 points).

Logistic Regression consistently achieves about 3.7–3.9 percentage points lower error than LDA. This is consistent with the findings in Problem 3. LDA's linear decision boundary assumption combined with the Gaussian class-conditional density assumption may not be well-suited for the SPAMBASE features, which are sparse word frequency counts that are far from normally distributed. Logistic Regression makes fewer distributional assumptions and is thus more flexible.

The choice of k (5 vs. 10) has minimal effect: increasing k slightly reduces the average error for both models (LR: $0.0759 \to 0.0737$; LDA: $0.1126 \to 0.1130$), as expected since larger k means more training data per fold. The fold-level variance is also slightly higher for $k = 10$ due to smaller validation sets.